

SAGE: A Multi-Source Retrieval-Augmented Knowledge Engine for Source-Grounded Question Answering over Websites, Documents, Videos, and Code Repositories

SRUJAN ZANJAL

Department of Computer Engineering
St. Vincent Pallotti College of Engineering and Technology
Nagpur, India
zanjalsrujan94@gmail.com

EXECUTIVE SUMMARY

SAGE is a prototype knowledge engine that converts websites, PDFs, YouTube videos, and public GitHub repositories into searchable knowledgebases with explicit citations and confidence-aware answers. The system unifies source-specific extraction pipelines with a shared retrieval-augmented generation architecture, backed by ChromaDB for vector search and Supabase PostgreSQL for provenance-rich metadata. A central design contribution is the use of deterministic *source overview chunks*, which improve broad questions such as summarisation and architecture explanation without invoking a second language model at indexing time. Prototype validation indicates that SAGE is suitable for major-project demonstration and prototype use, while OCR for scanned PDFs, private repository support, and stronger production hardening remain future work.

Abstract—Information work increasingly spans heterogeneous artefacts such as web pages, technical documents, software repositories, and video transcripts. In practice, users must switch tools and search modes across these modalities, and generic large language model interfaces often fail to preserve provenance or to disclose when evidence is weak. This paper presents SAGE (Semantic Analysis and Generation Engine), a multi-source retrieval-augmented knowledge system that ingests websites, PDFs, YouTube videos, and public GitHub repositories into modality-aware knowledgebases and answers user questions with citations, snippets, and heuristic confidence scores. SAGE employs source-specific extractors—Playwright for rendered websites, PyMuPDF for born-digital PDFs, transcript acquisition with a speech-to-text fallback for YouTube videos, and repository cloning or archive fallback for GitHub—followed by chunking, deterministic source-overview generation, sentence-transformer embeddings, vector indexing in ChromaDB, metadata persistence in Supabase, query rewriting, reranking, and answer generation. A shared metadata contract enables evidence rendering as URL-level, page-level, timestamp-level, and file-line citations. Prototype-level validation on representative sources shows that the architecture supports robust ingestion and question answering across all four modalities, and that overview chunks materially improve broad summarisation and repository or video overview questions while preserving provenance. The

system is ready for major-project demonstration and prototype use.

Index Terms—retrieval-augmented generation, semantic search, source-grounded question answering, vector database, knowledge engine, document intelligence, video intelligence, code intelligence

I. INTRODUCTION

Modern knowledge work is fragmented across multiple source types. A single learning or development task may require consulting a public website, a PDF specification, a long-form video, and a source-code repository. Traditional keyword search is effective for precise lexical lookup, but it is less effective for higher-level questions such as “What is this source about?”, “Which files define the API routes?”, or “What does the speaker conclude in the final section?” Dense retrieval and retrieval-augmented generation (RAG) have become practical ways to combine semantic retrieval with neural generation, reducing unsupported answers by grounding generation in retrieved evidence [1], [2], [5].

Despite these advances, most deployed assistants remain modality-specific. Document QA systems commonly focus on PDFs or visually rich forms; speech-oriented systems focus on transcription; repository assistants focus on code; and website assistants often ignore client-side rendering or provenance-rich citation requirements. Generic chatbot interfaces further complicate trust because they may provide fluent text without exposing where evidence came from or whether evidence is weak.

This paper presents **SAGE** (*Semantic Analysis and Generation Engine*), a unified prototype that brings four source types under one source-grounded QA architecture: websites, PDFs, YouTube videos, and public GitHub repositories. The system is implemented as a FastAPI backend with a React front end, ChromaDB vector storage, Supabase PostgreSQL metadata storage, and a website-focused Chrome side panel extension. SAGE provides two answer modes: *Grounded Mode*, which constrains responses to retrieved evidence, and *Exploratory*

Mode, which uses source evidence first and then allows carefully separated broader explanation.

A key design challenge in multi-source QA is the handling of broad or summarisation-oriented questions. Dense retrieval over only fine-grained chunks can yield weak evidence for questions such as “Summarise this website” or “Explain the repository architecture.” To address this, SAGE introduces *deterministic source overview chunks*: a single overview chunk is generated for every ingested knowledgebase from source-derived statistics, metadata, and excerpts, without invoking a language model at indexing time. These overview chunks improve the retrievability of broad questions and preserve provenance because each overview is traceable to the ingested source rather than to free-form model generation.

The main contributions of this work are:

- 1) a unified multi-source RAG architecture for website, PDF, video, and repository ingestion with source-specific citation metadata;
- 2) deterministic source overview chunks that improve broad question handling without adding a second generative stage at indexing time;
- 3) a prototype implementation with grounded and exploratory QA modes, confidence scoring, and a website-only Chrome side panel; and
- 4) a realistic prototype validation study that surfaces both strengths and limitations of the system across ingestion, retrieval, citation rendering, and source lifecycle operations.

II. RELATED WORK

RAG combines parametric generation with non-parametric retrieval, allowing a language model to condition responses on external evidence rather than only on memorised parameters [1]. Dense Passage Retrieval (DPR) showed that dual-encoder semantic retrieval can outperform classical sparse retrieval for open-domain question answering when trained on question-passage pairs [2]. These ideas are underpinned by Transformer-based representation learning [4] and by efficient sentence embedding methods such as Sentence-BERT, which make passage-level similarity search practical at application scale [3]. Recent surveys further emphasise provenance, updateability, and modularity as major drivers for real-world RAG deployment [5].

Vector search infrastructure is a critical enabling layer for such systems. FAISS demonstrated scalable nearest-neighbour search for high-dimensional embeddings [6], while newer application-oriented systems such as Chroma expose collection-based storage, metadata filtering, and query interfaces suited to developer workflows [11]. SAGE adopts this application-oriented model while keeping a separate relational metadata store for provenance, lifecycle state, and source management.

For documents, prior work such as LayoutLM showed that visually rich document understanding benefits from jointly modelling text and layout [7]. SAGE intentionally targets a narrower but highly practical subset of this problem: born-digital PDFs with selectable text. This design avoids the

cost and complexity of OCR in the core pipeline, while still leaving scanned-document support as future work. For speech and video, Whisper demonstrated robust multilingual speech recognition from large-scale weak supervision [8]. Practical systems often combine caption retrieval with local speech-to-text fallback, exactly the staged strategy used in SAGE [17]–[19].

Code intelligence introduces its own retrieval challenges because relevance depends not only on semantic similarity but also on file structure, line locality, and repository-level context. CodeBERT established strong learned representations for natural-language/code interactions [9], while RepoCoder showed that repository-level retrieval can substantially improve code tasks by supplying cross-file context [10]. SAGE does not synthesise code, but it borrows the central insight that repository-level context matters. This motivates the repository overview chunk and path- plus line-aware metadata used in the GitHub module.

The gap addressed by SAGE is therefore not the absence of modality-specific techniques, but the absence of a practical system that unifies these techniques under one retrieval, provenance, and interaction model. SAGE contributes an engineering-oriented synthesis that standardises metadata, answer rendering, and broad-question support across four heterogeneous source types.

III. PROBLEM FORMULATION

Let a user provide a source S belonging to a modality set

$$\mathcal{T} = \{\text{website, pdf, video, github}\}. \quad (1)$$

For a source of type $t \in \mathcal{T}$, the ingestion pipeline constructs a knowledgebase

$$K^{(t)} = (C^{(t)}, M^{(t)}), \quad (2)$$

where $C^{(t)} = \{c_0, c_1, \dots, c_n\}$ is the set of chunks and $M^{(t)}$ is the associated metadata store. The first chunk c_0 is reserved for a deterministic source overview,

$$c_0 = g_t(S), \quad (3)$$

where $g_t(\cdot)$ is a non-generative, source-derived overview function specific to modality t . The remaining chunks are produced by a modality-specific segmenter

$$\{c_1, \dots, c_n\} = h_t(S). \quad (4)$$

Each chunk is embedded by an encoder f_θ ,

$$e_i = f_\theta(c_i), \quad e_q = f_\theta(q'), \quad (5)$$

where q' is the query after optional rewriting. Chunks are stored in a vector index together with metadata such as `source_id`, `knowledgebase_id`, `source_type`, `chunk_index`, and modality-specific provenance fields (URL, page number, timestamp, file path, and line range).

Given a user query q and selected knowledgebase $K^{(t)}$, the goal is to return an answer A and citation set Π such that:

- 1) A is relevant to q ;

- 2) A is grounded in retrieved evidence $\tilde{C} \subseteq C^{(t)}$;
- 3) each citation $\pi \in \Pi$ points to explicit provenance metadata in $M^{(t)}$; and
- 4) unsupported claims are suppressed or reflected as low confidence.

A practical objective is therefore

$$\max_{A, \Pi} \text{Rel}(A, \tilde{C}) + \eta \text{Prov}(\Pi, \tilde{C}) - \mu \text{Hall}(A, \tilde{C}), \quad (6)$$

where Rel rewards relevance, Prov rewards explicit provenance, and Hall penalises unsupported content. In SAGE, this objective is approximated through heuristics on retrieval score, reranker score, citation availability, and mode-specific answer policies rather than through end-to-end supervised training.

IV. SAGE ARCHITECTURE

SAGE is implemented as a layered architecture with distinct ingestion-time and query-time components. The front end provides source ingestion, source history, chat interaction, answer rendering, and a website-only browser side panel. The backend exposes ingestion, job tracking, question answering, and source-management endpoints. ChromaDB stores chunk vectors and supports similarity search, while Supabase PostgreSQL stores provenance metadata and lifecycle state.

The ingestion side of the architecture is modality-specific only up to extraction and chunk formation. After extraction, all modules converge to a shared indexing contract with three important properties: one knowledgebase per ingested source with a stable source identifier, one overview chunk plus multiple detailed chunks, and provenance-rich metadata for every chunk. This common representation allows the QA layer to remain largely modality-agnostic while still rendering citations in modality-aware form.

The extension subsystem is intentionally narrower than the main web application. It uses Chrome Manifest V3 and the `chrome.sidePanel` API to provide website QA directly inside the browser for public pages [21], [22]. Restricting the extension to website mode reduces permission scope and avoids complex file or external-service flows inside the browser package.

V. METHODOLOGY

A. Website Ingestion

The website module accepts a public HTTP(S) URL, normalises it, and launches a constrained crawler using Playwright [13]. Playwright is used rather than raw HTTP fetching because many modern pages depend on client-side rendering and dynamic DOM updates. The crawler follows same-domain links up to configured `max_depth` and `max_pages` limits. Extracted page content is cleaned to remove low-value boilerplate where possible, then segmented into chunks associated with URL-level metadata such as canonical reference, original reference, title, page index, and chunk index.

The website overview chunk is generated deterministically from crawl metadata. Its content may include the canonical URL, number of pages crawled, page titles, high-value URLs,

beginning and ending excerpts, and repeated informative terms. Because broad questions such as “What is this website about?” often do not match a single local paragraph, the overview chunk acts as a high-recall summary anchor while still remaining source-derived.

B. PDF Ingestion

The PDF module targets born-digital PDFs with selectable text. PyMuPDF is used for text extraction because it provides high-performance access to document text, block structure, and page-level information [14], [15]. During ingestion, text is extracted page by page and segmented into chunks that preserve `page_number` metadata. SAGE intentionally rejects empty or textless PDFs in the base pipeline, because the prototype focuses on reliable searchable text rather than OCR-heavy processing. PyMuPDF exposes OCR support through separate mechanisms [16], but OCR is intentionally left outside the main prototype to keep the baseline pipeline stable and transparent.

The PDF overview chunk contains deterministic information such as filename or title, page count, first-page excerpt, last-page excerpt, and chunk statistics. Detailed chunks remain page-grounded, enabling answer rendering with page-level citations.

C. Video Ingestion

The video module ingests public YouTube URLs. It first attempts transcript acquisition via `youtube-transcript-api`, which can retrieve existing human or automatically generated subtitles without requiring a browser or API key [17]. If no captions are available, the system can invoke an explicit fallback path in which `yt-dlp` fetches audio and `faster-whisper` performs local speech-to-text transcription [8], [18], [19]. For demonstration use, an already analysed video can be reused through video identifier matching, avoiding repeated transcription when a ready source already exists.

The transcript is converted into timestamped chunks, each associated with `start_time`, `end_time`, a human-readable `timestamp_label`, and the source URL or video identifier. The overview chunk summarises title, duration, transcript origin, beginning excerpts, ending excerpts, and coarse recurring themes using only transcript-derived evidence. This overview is especially valuable for questions such as “What is this video about?” or “Summarise the main points.”

D. GitHub Repository Ingestion

The GitHub module accepts public repository URLs and validates them strictly before any job is started. This prevents malformed or non-repository URLs from entering the pipeline. When valid, the module attempts a shallow clone and falls back to archive download if needed. Repository files are filtered by type and size to skip binaries, extremely large artefacts, and non-informative generated content. Text files are chunked by line ranges, preserving metadata fields such as

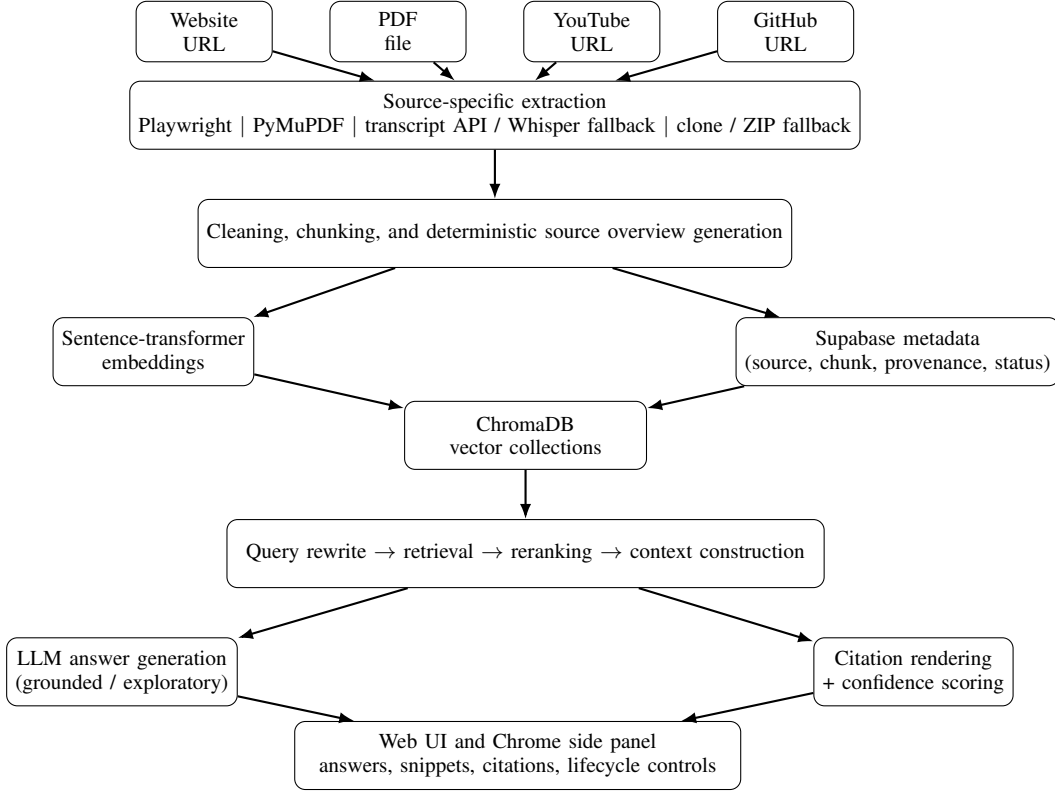


Fig. 1. System architecture of SAGE. Four heterogeneous sources are normalised into a common chunk-plus-metadata representation, indexed in ChromaDB and Supabase, and queried through a shared retrieval, reranking, and citation-aware answer pipeline.

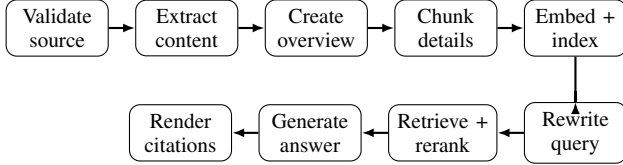


Fig. 2. Index-time and query-time timeline used across SAGE modules.

file_path, start_line, end_line, inferred language, and chunk index.

The repository overview chunk is generated from repository metadata rather than from free-form summarisation. Typical fields include repository name and branch, detected languages, significant files such as README, requirements.txt, Dockerfile, top-level directories, and likely entry-point files. This chunk improves architecture-level and onboarding questions, while detailed file chunks support precise implementation questions. Importantly, the GitHub module never executes code; it only indexes textual repository contents.

E. Shared Embedding, Retrieval, and QA Modes

All chunks are embedded using sentence-transformers/all-MiniLM-L6-v2, a compact sentence and short-paragraph encoder intended for information retrieval and semantic similarity [20]. Embeddings and chunk documents are stored in Chroma

collections [11], while metadata and lifecycle state are stored in Supabase PostgreSQL tables [12]. This split allows the system to combine efficient vector search with structured provenance-rich metadata.

At query time, the system optionally rewrites the question to improve retrieval specificity, embeds the rewritten question, retrieves top- k candidate chunks, reranks them, constructs a context window, and calls a hosted language model API for answer generation. *Grounded Mode* is conservative: it prefers explicit evidence and will down-rank or refuse answers when evidence is weak. *Exploratory Mode* still retrieves source evidence first, but may add broader explanation so long as source-backed content is clearly represented first.

VI. ALGORITHMS AND SCORING

Algorithm 1 abstracts the common ingestion path. Modality-specific behaviour appears in EXTRACT and CHUNKDETAILED, while overview generation is standardised across all modules.

Algorithm 2 is deliberately non-generative. The overview chunk is deterministic with respect to extracted evidence and metadata, which avoids introducing a second hallucination channel during indexing.

Retrieval begins with cosine similarity between query and chunk embeddings,

$$s_i = \cos(e_q, e_i) = \frac{e_q^\top e_i}{\|e_q\| \|e_i\|}. \quad (7)$$

Algorithm 1 Multi-source ingestion and indexing

Require: source descriptor S , source type t **Ensure:** knowledgebase identifier K or structured error

```

1: validate and normalise  $S$  for modality  $t$ 
2:  $E \leftarrow \text{EXTRACT}(S, t)$ 
3: if  $E$  is empty or unusable then
4:   return structured ingestion error
5: end if
6:  $m \leftarrow \text{COLLECTMETA}(S, E, t)$ 
7:  $c_0 \leftarrow \text{GENERATEOVERVIEW}(E, m, t)$ 
8:  $D \leftarrow \text{CHUNKDETAILED}(E, m, t)$ 
9:  $C \leftarrow [c_0] \cup D$ 
10: for all  $c_i \in C$  do
11:    $e_i \leftarrow f_\theta(c_i)$ 
12:   write  $(c_i, e_i)$  to ChromaDB
13:   write provenance metadata of  $c_i$  to Supabase
14: end for
15: mark source status as ready
16: return knowledgebase identifier  $K$ 

```

Algorithm 2 Deterministic source overview chunk generation

Require: extracted evidence E , metadata m , modality t **Ensure:** overview chunk c_0

```

1: initialise template  $T_t$  for modality  $t$ 
2: compute basic statistics: counts, duration or page count,
   titles, key files, URLs
3: collect early and late excerpts from  $E$ 
4: compute repeated informative terms or headings when
   available
5: populate  $T_t$  with only source-derived fields
6: set metadata:
   chunk_kind = source_overview
   is_sage_overview = true
   overview_type = t
   chunk_index = 0
7: return compiled overview chunk  $c_0$ 

```

SAGE then applies a lightweight reranking stage that combines semantic similarity with lexical overlap, metadata relevance, and an overview bonus for broad queries:

$$r_i = \alpha s_i + \beta \text{lex}(q', c_i) + \gamma \text{meta}(q', m_i) + \delta b(q') \mathbb{I}[c_i \text{ is overview}] \quad (8)$$

where $b(q')$ is a broad-query gate and $\alpha + \beta + \gamma + \delta = 1$ for interpretability. Equation (8) expresses the system design rather than a learned global model.

Confidence is represented as a heuristic score rather than a calibrated posterior probability. A practical approximation used for answer display is

$$\kappa = \text{clip}_{[0,1]} (\lambda_1 \bar{r} + \lambda_2 \log(1 + |\Pi|) + \lambda_3 a - \lambda_4 u), \quad (9)$$

where $\bar{r}$ is the mean reranked score of retained evidence, $|\Pi|$ is citation count, a is evidence agreement, and u is an unsupported-claim penalty. This design makes confidence

Algorithm 3 Source-grounded question answering with citations

Require: query q , selected knowledgebase K , mode $m \in \{\text{grounded}, \text{exploratory}\}$ **Ensure:** answer A , citation set Π , confidence κ

```

1:  $q' \leftarrow \text{REWRITEQUERY}(q, K)$ 
2: retrieve top- $k$  chunks  $R \leftarrow \text{RETRIEVE}(q', K)$ 
3:  $\tilde{R} \leftarrow \text{RERANK}(R, q', K)$ 
4: if  $m = \text{grounded}$  and evidence is below threshold then
5:   return cautious refusal or low-confidence answer with
     weak-evidence note
6: end if
7:  $X \leftarrow \text{BUILDCONTEXT}(\tilde{R}, m)$ 
8:  $A \leftarrow \text{GENERATEANSWER}(q', X, m)$ 
9:  $\Pi \leftarrow \text{RENDERCITATIONS}(\tilde{R})$ 
10:  $\kappa \leftarrow \text{ESTIMATECONFIDENCE}(\tilde{R}, \Pi, A, m)$ 
11: return  $(A, \Pi, \kappa)$ 

```

interpretable as operational guidance for the user rather than as a strict probabilistic calibration.

VII. PROTOTYPE VALIDATION AND RESULTS

The evaluation of SAGE is a *prototype validation*, not a benchmark leader-board experiment. The goal is to verify that the implemented system ingests sources correctly, retrieves relevant evidence, renders accurate citations, handles out-of-scope queries conservatively, and supports complete user workflows including delete and re-ingest operations.

Representative sources used during validation include the *About Python* website, a compact born-digital two-page PDF fixture for document validation, a short TED YouTube video for transcript and timestamp testing, and the public repository `0xTheProDev/fastapi-clean-example` for GitHub ingestion. The validation suite combined backend tests, API-level checks, representative user questions, citation verification, and front-end interactions.

Table I summarises the four source modules and their provenance behaviour. A major engineering goal of SAGE is that each modality preserves the citation features that are natural to its source type, rather than flattening all evidence into anonymous text fragments.

Table II shows that the prototype is strongest in provenance-preserving QA and lifecycle reliability, while its main limitations arise from external dependencies: browser automation for websites, OCR omission for scanned PDFs, local speech-to-text cost for videos, and hosted LLM rate limits for repeated requests.

Citation validation focused on whether snippets actually support the returned answer and whether provenance pointers are complete. For website and PDF modules, citations correctly mapped back to page or URL-level evidence. For the video module, timestamp labels and start/end times made it possible to connect an answer to a specific transcript span. For the GitHub module, file-path and line-range citations supported direct inspection of repository evidence.

TABLE I
MODULE CAPABILITY COMPARISON IN THE SAGE PROTOTYPE.

Module	Input	Extraction method	Citation type	Overview chunk	Prototype status
Website	Public URL	Playwright-rendered crawl with same-domain depth limits	URL/snippet citation with source reference and chunk index	Website overview	Ready for demo
PDF	Selectable-text PDF	PyMuPDF page-wise extraction and chunking	Page-level citation with page number and snippet	PDF overview	Ready for demo
Video	Public YouTube URL	Transcript API first; yt-dlp + faster-whisper fallback	Timestamp citation with start/end time and snippet	Video overview	Ready for demo [†]
GitHub	Public repository URL	Shallow clone or ZIP fallback with file filtering and line-range chunking	File-path and line-range citation	Repository overview	Ready for demo

[†]Pre-analysed videos are preferred in live demonstrations because local auto-transcription can be slow on CPU.

TABLE II
APPROXIMATE PROTOTYPE VALIDATION SUMMARY. VALUES REFLECT REPRESENTATIVE MODULE-LEVEL TESTS RATHER THAN A STANDARDISED BENCHMARK.

Module	Ingestion validation	QA validation	Citation checks	Front-end checks	Primary limitation
Website	7 scenarios validated; low-text handling improved	≈ 9/10 representative questions satisfactory	5/5 representative checks passed	6/6 checks passed	JavaScript-heavy or anti-bot sites may fail
PDF	Valid, empty, unsupported-file, and lifecycle paths verified; image-only path code-verified	9/10 questions satisfactory, one partial out-of-scope case	5/5 checks passed	7/7 checks passed	OCR not in core pipeline
Video	Direct transcript path plus explicit fallback path validated; source reuse added	8 pass, 1 provider-limit partial, 1 safe out-of-scope behaviour	5/5 timestamp checks passed	8/8 checks passed	Auto-transcription latency on CPU
GitHub	URL validation, ingestion, and overview-generation suites passed; sequential stability improved	≈ 9/10 sequential QA stability after fixes	File-line citations validated on representative questions	UI flow checked / code-verified	Public repositories only; provider rate limits

TABLE III
CITATION SCHEMA USED BY EACH MODALITY.

Modality	Representative citation fields
Website	source_ref, snippet, score, chunk_index
PDF	source_ref, page_number, snippet, score, chunk_index
Video	video_id, timestamp_label, start_time, end_time, snippet, score
GitHub	file_path, start_line, end_line, snippet, score

The most notable qualitative improvement came from broad-question handling. Prior to overview chunk insertion, several broad questions either retrieved weakly connected local chunks or forced the answer generator to refuse a response in grounded mode. After overview chunk standardisation, the system produced more stable high-level answers with explicit provenance, especially for video and repository questions where the semantic gap between a global question and local

chunks is large.

VIII. DISCUSSION, RELIABILITY, AND FUTURE WORK

A. Discussion

The core design lesson from SAGE is that *provenance is not a rendering afterthought*. Provenance must be preserved from ingestion onwards through metadata design, index persistence, reranking, and answer rendering. URL-only citations are insufficient for PDFs, timestamps are indispensable for video QA, and file-line citations are essential for code intelligence. By standardising the chunk metadata contract while keeping modality-specific fields, SAGE achieves both architectural reuse and interface specificity.

A second lesson is that broad-question support is a retrieval problem rather than merely a prompting problem. The overview chunk design increases the chance that the retrieval stage returns a high-level, source-derived representation before generation occurs. This reduces the temptation to compensate for retrieval weakness with larger prompts or stronger language-model speculation. In that sense, overview chunks act as a structured retrieval prior.

TABLE IV
OBSERVED BENEFITS OF DETERMINISTIC OVERVIEW CHUNKS FOR BROAD QUESTIONS.

Module	Representative broad question	Without overview chunk	With overview chunk
Website	“What is this website about?”	Retrieval may favour local paragraphs that are individually relevant but globally incomplete	Broader page summary becomes retrievable while still pointing to source URLs and snippets
PDF	“Summarise this PDF”	Chunk-level retrieval may over-emphasise a single page or section	Overview chunk improves whole-document retrieval while page-level chunks remain available for detail
Video	“What is this video about?”	Fine-grained transcript retrieval can yield weak evidence for global summarisation	Video overview plus timestamped chunks improves initial grounding and answerability
GitHub	“Explain the overall architecture”	Retrieval may surface isolated files with insufficient repository-level context	Repository overview exposes important files, directories, and entry points for architecture questions

B. Security and Reliability Considerations

SAGE is designed as a prototype, but several reliability measures are already present. Input validation rejects malformed URLs and unsupported file types early in the API layer. The GitHub module does not execute repository code and only indexes filtered text files, which reduces risk when ingesting arbitrary public repositories. The PDF module rejects empty or unusable files instead of silently producing empty knowledgebases. The video module explicitly distinguishes between transcript unavailability and automatic transcription fallback.

The architecture also reveals a common distributed-systems issue in hybrid RAG pipelines: metadata-vector consistency. Because vectors and relational metadata are written to different stores, partial failures can yield stale records where a source appears ready in metadata but has incomplete vector coverage. SAGE mitigates this through source lifecycle management, delete/re-ingest workflows, and targeted regression tests, but stronger transactional guarantees remain an open engineering improvement.

External-service dependence also affects reliability. Hosted LLM APIs can rate-limit repeated requests, which was observed in the sequential GitHub and video validation runs. Browser automation depends on Playwright and a matching Chromium installation. Local speech-to-text performance depends on hardware and on the selected Whisper-compatible model size. Consequently, the prototype is best described as *ready for major-project demonstration and prototype use*, not as a fully production-hardened system.

C. Limitations

The present system does not include user authentication or multi-user isolation. It assumes a prototype deployment where a single operator or small team manages ingestion state. OCR for scanned, image-only PDFs is omitted from the core pipeline to prioritise reliability on selectable text. Video ingestion depends on the availability of captions or on potentially slow local speech-to-text. GitHub processing is limited to public repositories and bounded by file and chunk caps to prevent unmanageable ingest times. Finally, confidence

TABLE V
PRIMARY LIMITATIONS AND CURRENT MITIGATIONS.

Limitation	Impact	Current mitigation
Scanned PDFs without OCR	Missing document coverage	Reject unusable files; mark OCR as future work
CPU-bound auto-transcription	Slow live ingestion	Prefer pre-analysed short videos and source reuse
Public GitHub only	No private repository support	Restrict prototype scope to public repositories
Provider rate limits	Partial QA interruptions	Conservative timeouts, retries, and partial-result handling
Vector-metadata drift	Empty retrieval for stale sources	Delete/re-ingest workflow and lifecycle checks
JS-heavy or protected sites	Crawl failure	Playwright rendering and demo-safe crawl limits

values are heuristic and useful for user guidance, but they should not be interpreted as calibrated probabilities.

D. Future Work

Several improvements follow naturally from the prototype. First, OCR integration would extend the PDF module to scanned documents and visually rich layouts, potentially with layout-aware models for richer provenance. Second, private GitHub repository access and authenticated website ingestion would broaden enterprise applicability. Third, production deployment would benefit from a stronger job queue and worker architecture, for example via Redis and Celery, to isolate long-running video and crawl workloads. Fourth, hybrid retrieval that combines semantic vectors with stronger lexical retrieval could further improve precision on code identifiers, exact citations, and structured documents. Finally, a larger benchmark study with controlled question sets, annotation guidelines,

and calibrated faithfulness metrics would convert the current prototype validation into a more formal evaluation framework.

IX. CONCLUSION

This paper presented SAGE, a multi-source retrieval-augmented knowledge engine designed to convert websites, PDFs, YouTube videos, and public GitHub repositories into source-grounded conversational knowledgebases. The system unifies modality-specific extraction with a shared indexing, retrieval, reranking, and provenance architecture, while preserving source-appropriate citations such as URL references, page numbers, timestamps, and file-line spans. A central contribution is the introduction of deterministic source overview chunks, which improve broad summarisation and architecture-oriented questions without adding a second generative stage at indexing time. Prototype validation indicates that SAGE is a practical and technically coherent basis for major-project demonstration and further research on unified, provenance-aware AI assistants.

REFERENCES

- [1] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *Advances in Neural Information Processing Systems*, vol. 33, 2020, pp. 9459–9474.
- [2] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, “Dense Passage Retrieval for Open-Domain Question Answering,” in *Proc. EMNLP*, 2020, pp. 6769–6781.
- [3] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention Is All You Need,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 5998–6008.
- [5] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, “Retrieval-Augmented Generation for Large Language Models: A Survey,” *arXiv preprint arXiv:2312.10997*, 2023.
- [6] J. Johnson, M. Douze, and H. Jégou, “Billion-scale Similarity Search with GPUs,” *arXiv preprint arXiv:1702.08734*, 2017.
- [7] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” in *Proc. ACM SIGKDD*, 2020, pp. 1192–1200.
- [8] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, “Robust Speech Recognition via Large-Scale Weak Supervision,” in *Proc. ICML*, vol. 202, 2023, pp. 28492–28518.
- [9] Z. Feng, D. Guo, D. Tang, N. Duan, X. Feng, M. Gong, L. Shou, B. Qin, T. Liu, D. Jiang, and M. Zhou, “CodeBERT: A Pre-Trained Model for Programming and Natural Languages,” in *Findings of EMNLP*, 2020, pp. 1536–1547.
- [10] F. Zhang, B. Chen, Y. Zhang, J. Keung, J. Liu, D. Zan, Y. Mao, J.-G. Lou, and W. Chen, “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation,” in *Proc. EMNLP*, 2023, pp. 2471–2484.
- [11] Chroma, “Chroma documentation,” 2026. [Online]. Available: <https://docs.trychroma.com>. Accessed: May 10, 2026.
- [12] Supabase, “Database overview,” 2026. [Online]. Available: <https://supabase.com/docs/guides/database/overview>. Accessed: May 10, 2026.
- [13] Microsoft, “Playwright documentation,” 2026. [Online]. Available: <https://playwright.dev/>. Accessed: May 10, 2026.
- [14] PyMuPDF, “PyMuPDF documentation,” 2026. [Online]. Available: <https://pymupdf.readthedocs.io/>. Accessed: May 10, 2026.
- [15] PyMuPDF, “Text extraction recipes,” 2026. [Online]. Available: <https://pymupdf.readthedocs.io/en/latest/recipes-text.html>. Accessed: May 10, 2026.
- [16] PyMuPDF, “OCR – Optical Character Recognition,” 2026. [Online]. Available: <https://pymupdf.readthedocs.io/en/latest/recipes-ocr.html>. Accessed: May 10, 2026.
- [17] jdepoix, “YouTube Transcript API,” GitHub repository, 2026. [Online]. Available: <https://github.com/jdepoix/youtube-transcript-api>. Accessed: May 10, 2026.
- [18] yt-dlp, “yt-dlp,” GitHub repository, 2026. [Online]. Available: <https://github.com/yt-dlp/yt-dlp>. Accessed: May 10, 2026.
- [19] SYSTRAN, “faster-whisper,” GitHub repository, 2026. [Online]. Available: <https://github.com/SYSTRAN/faster-whisper>. Accessed: May 10, 2026.
- [20] Sentence Transformers, “all-MiniLM-L6-v2 model card,” 2026. [Online]. Available: <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>. Accessed: May 10, 2026.
- [21] Chrome for Developers, “chrome.sidePanel API,” 2026. [Online]. Available: <https://developer.chrome.com/docs/extensions/reference/api/sidePanel>. Accessed: May 10, 2026.
- [22] Chrome for Developers, “Manifest V3 overview,” 2026. [Online]. Available: <https://developer.chrome.com/docs/extensions/develop/migrate/what-is-mv3>. Accessed: May 10, 2026.